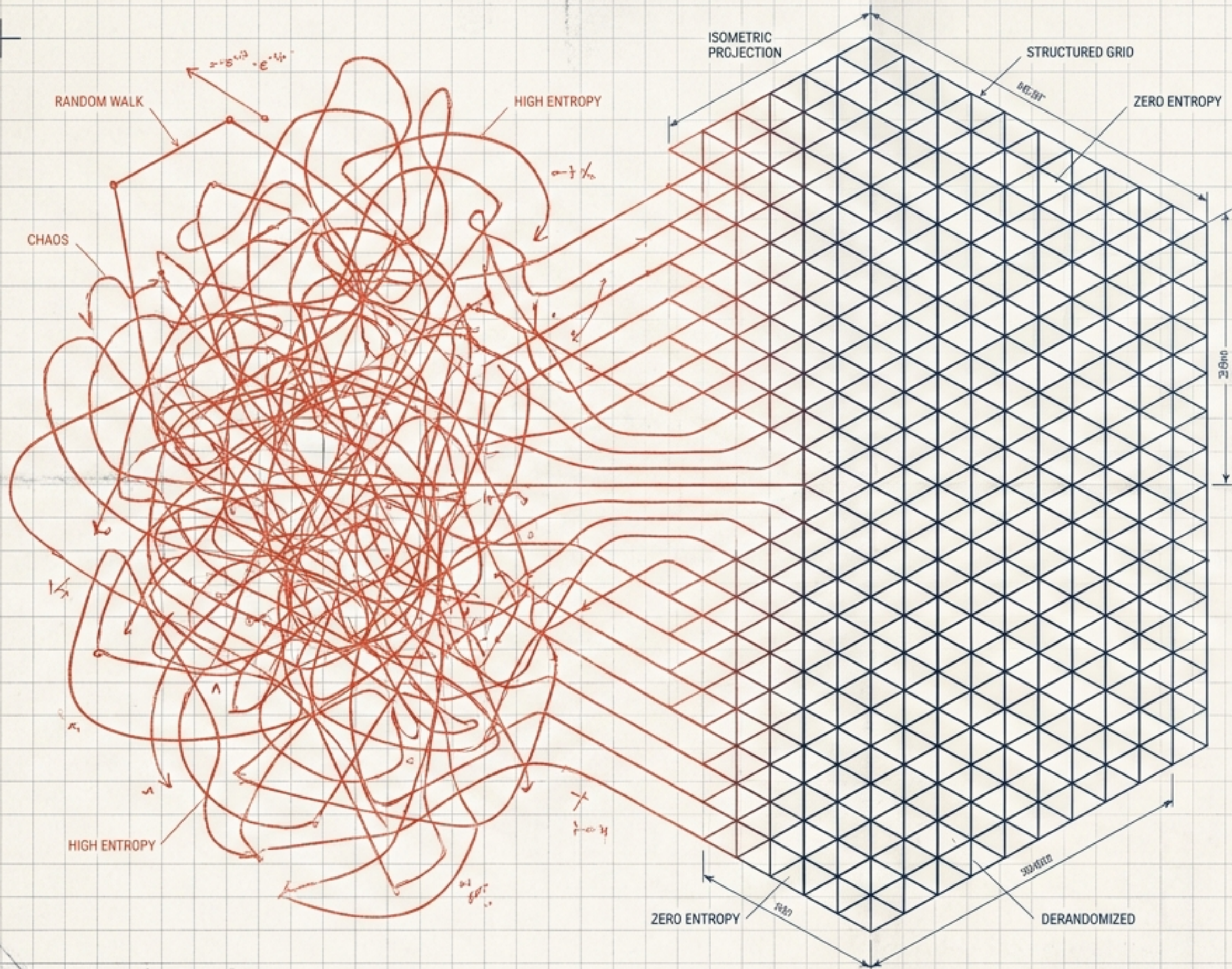


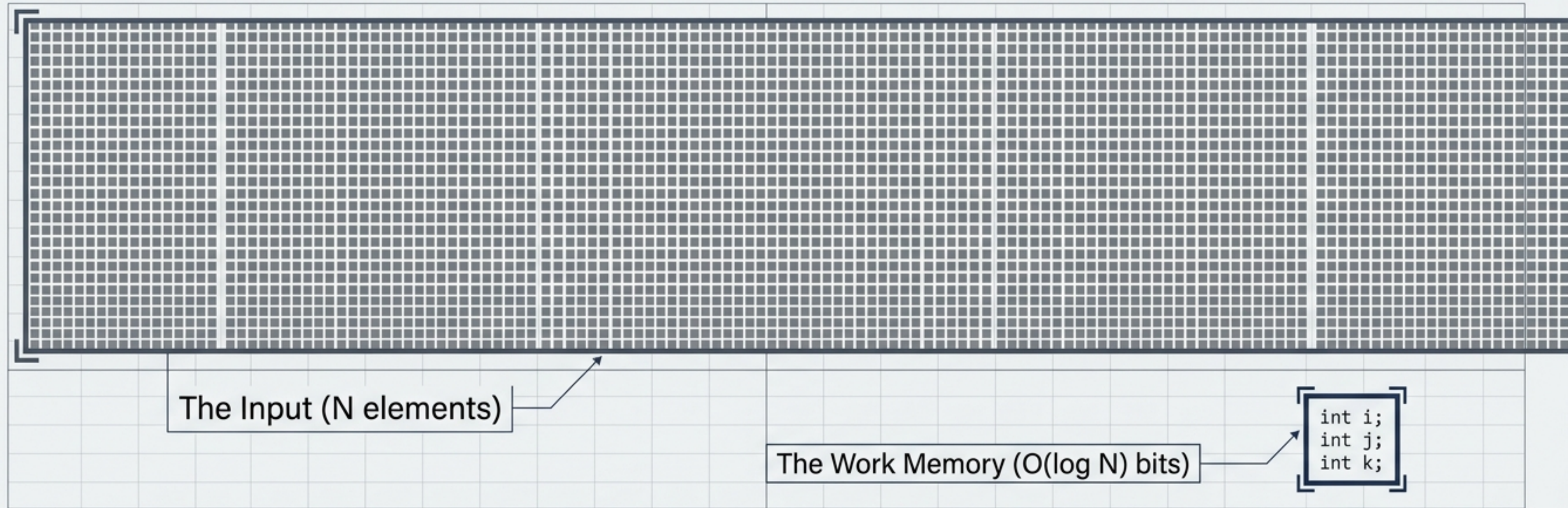


Exact Derandomization of Logarithmic Space

How a 108-page
breakthrough proved
 $L = RL = BPL$

A Technical Deconstruction
for Software Engineers

COMPUTATIONAL COMPLEXITY THEORY: RANDOMIZED LOGARITHMIC SPACE



The Core Question: If a computer only has enough RAM to hold a few index pointers, does giving it a random number generator fundamentally increase its computational power?

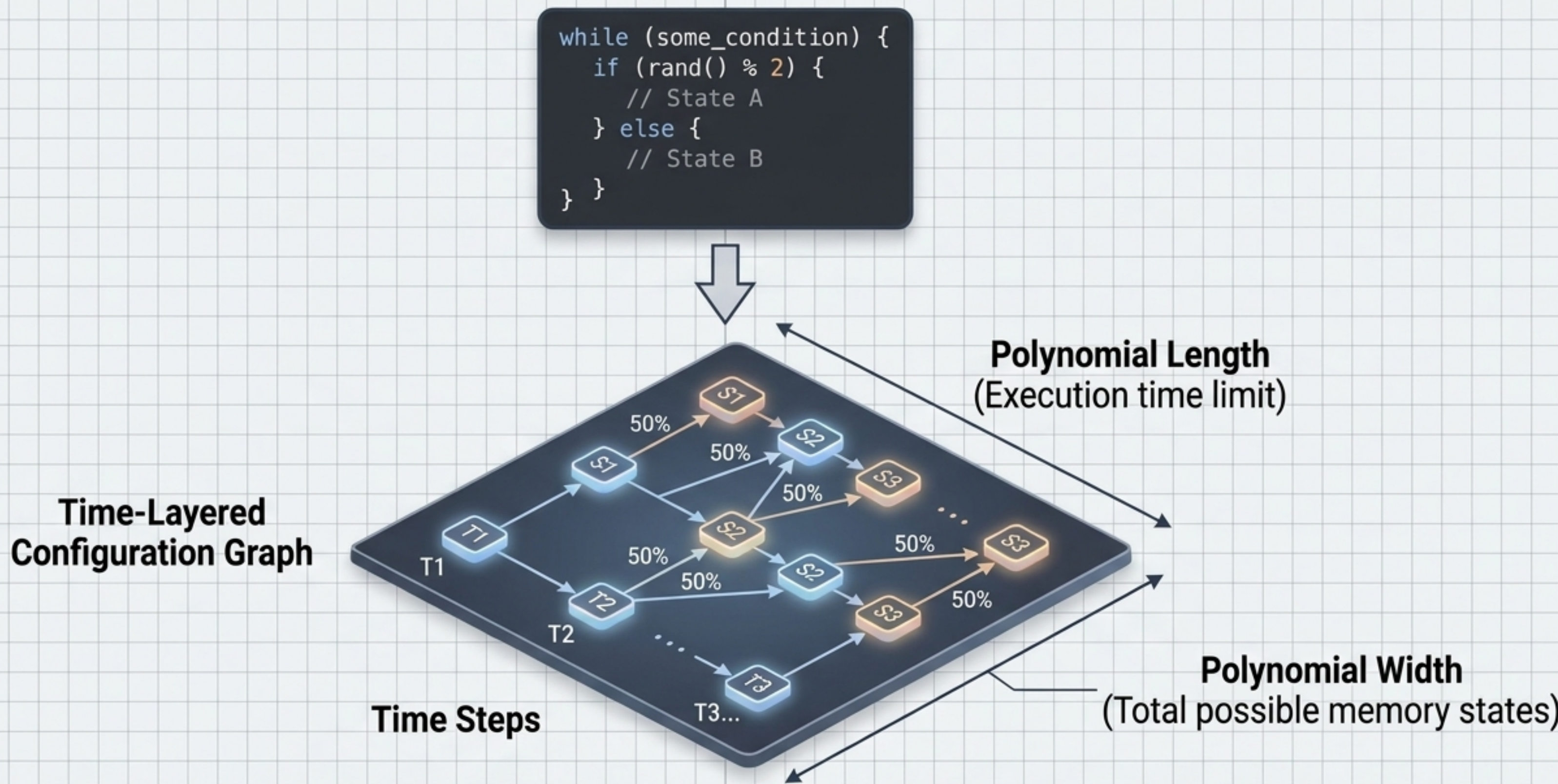
Key Insight: A logarithmic-space machine can remember a bounded number of positions in its input, but cannot keep a polynomial-length computation or random tape.

COMPUTATIONAL COMPLEXITY THEORY: COMPARING LOGARITHMIC SPACE CLASSES

Class	Memory Allowed	Randomness?	Win Condition (Yes)	Fail Condition (No)
Deterministic Log Space (L)	$O(\log N)$ (Few Pointers)	None	Always Accepts	Always Rejects
Randomized Log Space (RL)	$O(\log N)$	Fair Coins	Prob $\geq 1/2$	Prob = 0 (No false positives)
Bounded-Error Probabilistic (BPL)	$O(\log N)$	Fair Coins	Prob $\geq 2/3$	Prob $\leq 1/3$ (Two-sided error)

Takeaway: All randomized classes enforce a strict polynomial time bound to prevent infinite coin-flipping loops.

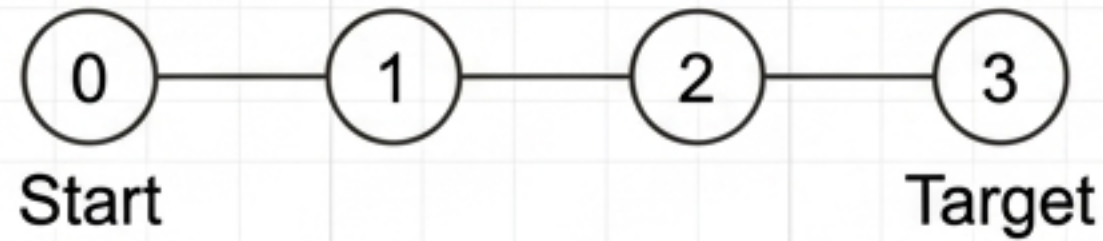
COMPUTATIONAL COMPLEXITY THEORY: TRANSLATING SOFTWARE TO STRUCTURE



Fixing the input turns the machine into a **finite, probabilistic maze**.

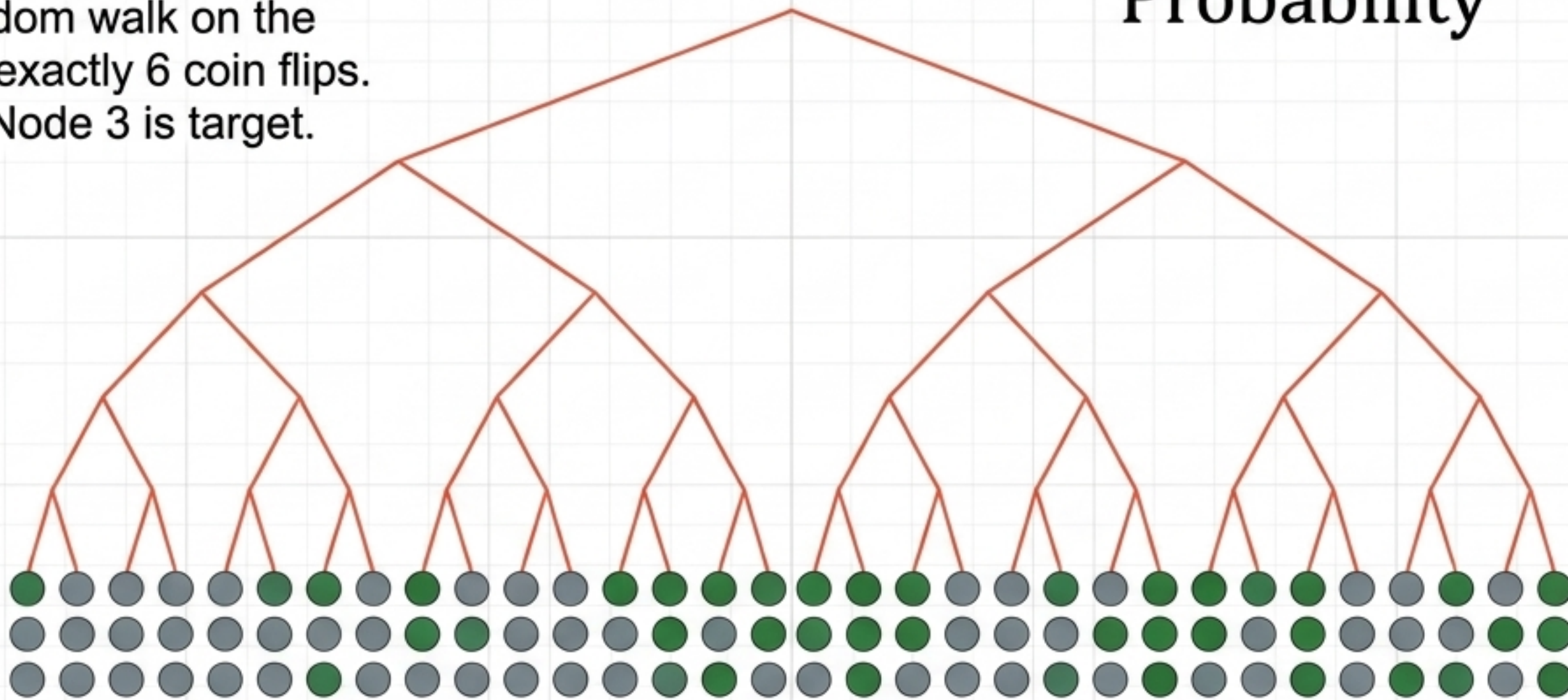
COMPUTATIONAL COMPLEXITY THEORY:

DERANDOMIZATION: CALCULATING ACCEPTANCE PROBABILITY



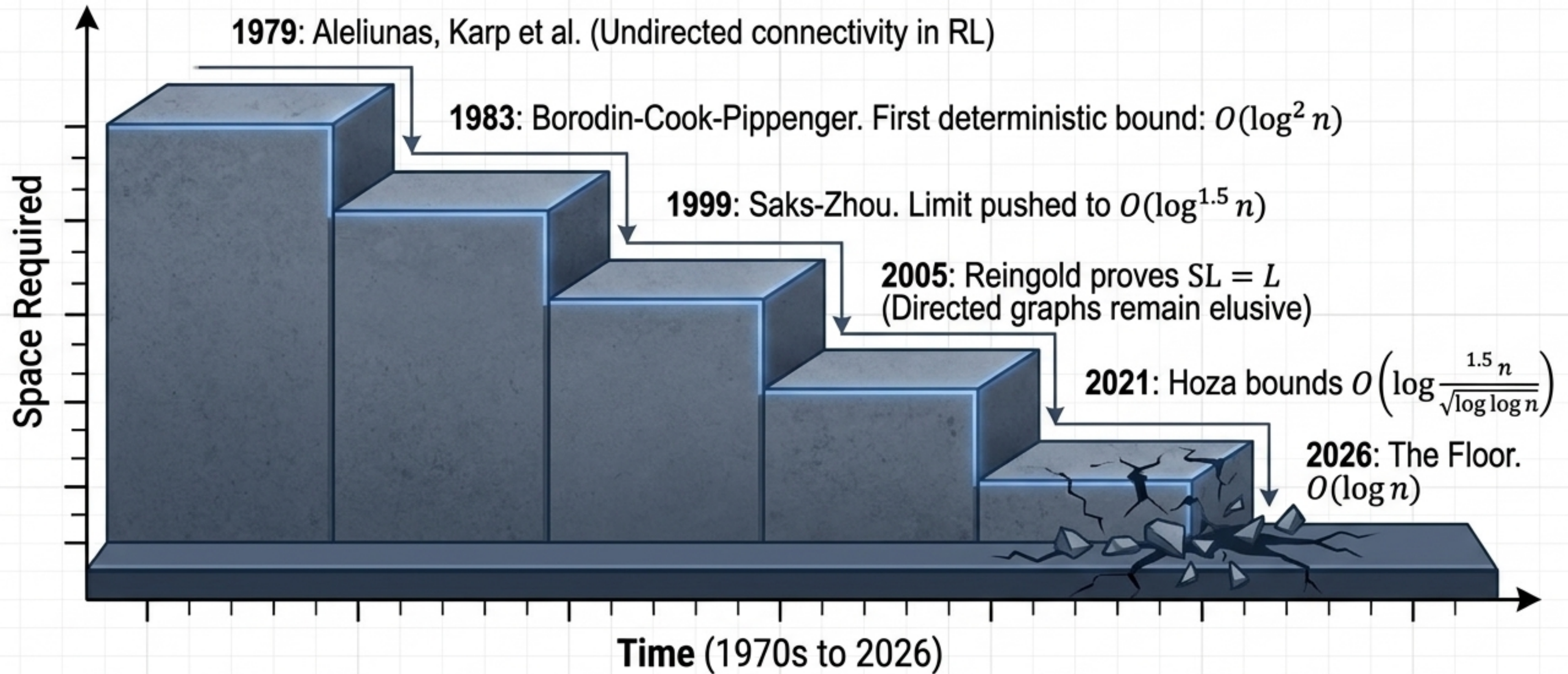
Scenario: A random walk on the path 0-1-2-3 for exactly 6 coin flips. Node 0 is start, Node 3 is target.

$$\text{Acceptance Probability} = \frac{28}{64} = \frac{7}{16}$$



Concept: To “derandomize” this without infinite memory, we must precisely calculate this 7/16 probability deterministically.

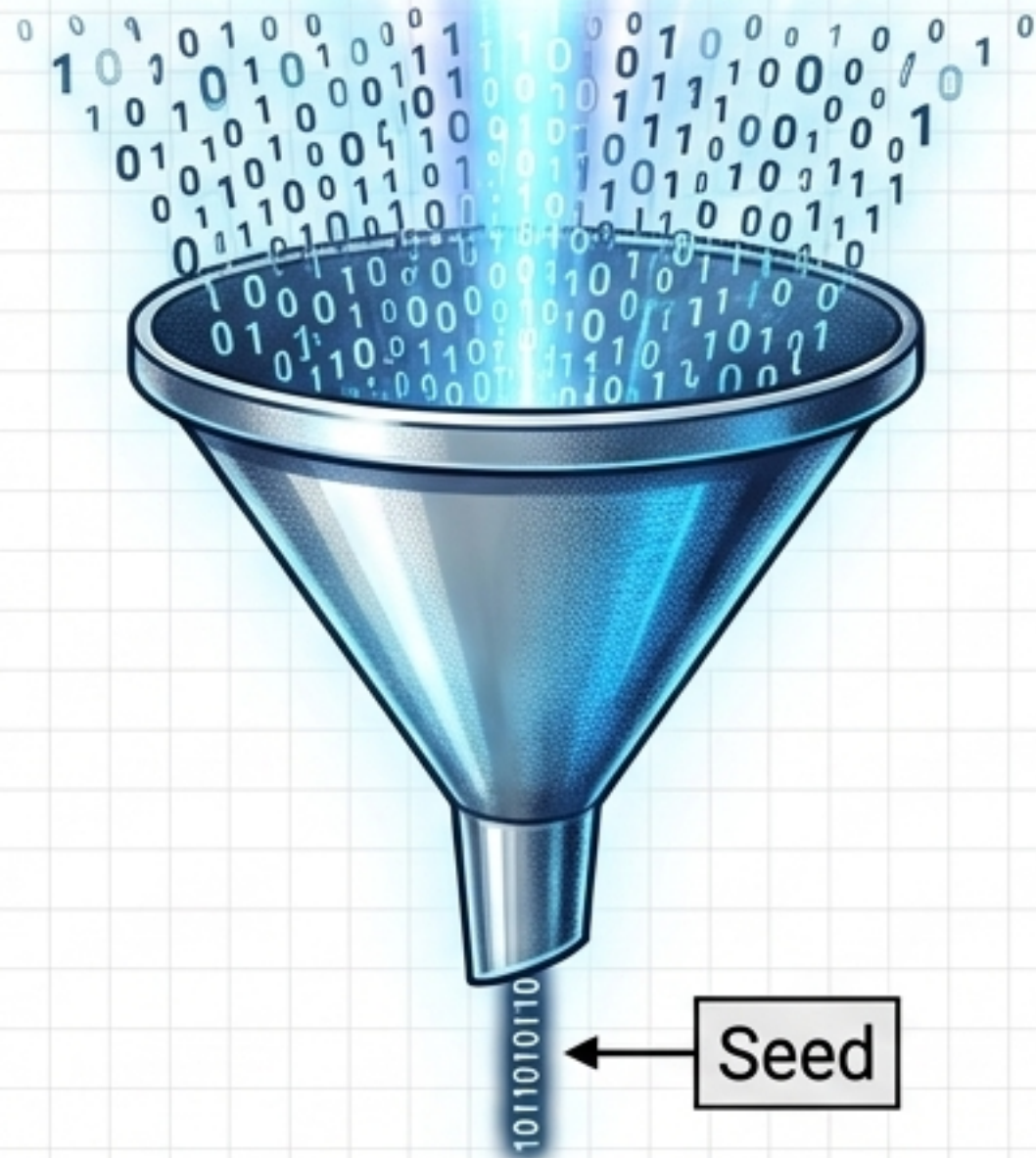
COMPUTATIONAL COMPLEXITY THEORY: HISTORICAL TIMELINE OF SPACE COMPLEXITY IN GRAPH CONNECTIVITY



The ultimate goal is to reach the absolute theoretical floor of logarithmic space for directed connectivity.

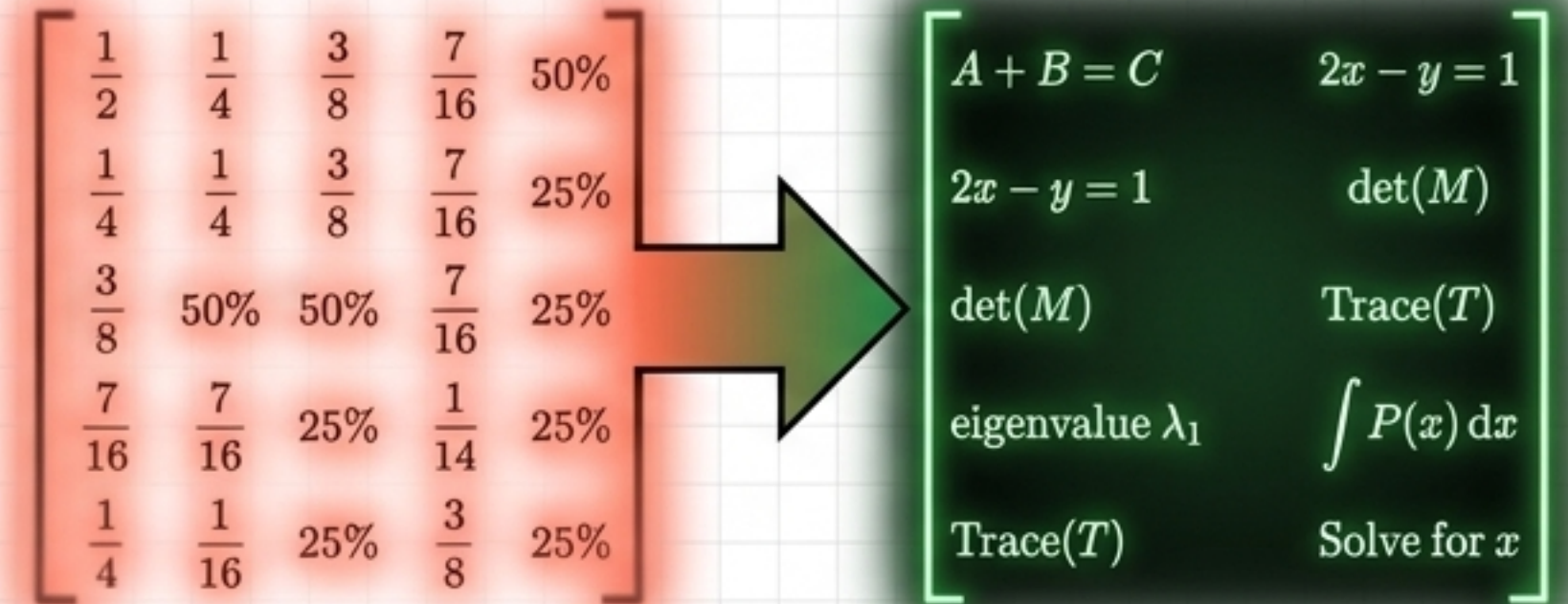
COMPUTATIONAL COMPLEXITY THEORY: DERANDOMIZATION PARADIGMS

The Old Paradigm: Pseudorandom Generators



Shrink a long random string into a short seed using $O(\log^2 n)$ memory. Evaluates paths, but still requires too much space.

The New Paradigm: Exact Derandomization



Convert the entire probabilistic maze into a deterministic reward-calculation matrix. No random bits required for the core calculation.

Exact Derandomization of Logarithmic Space: $L = RL = BPL$

OpenAI, September 2026 | 108 pages

S = The strictly forward substochastic matrix (the maze).

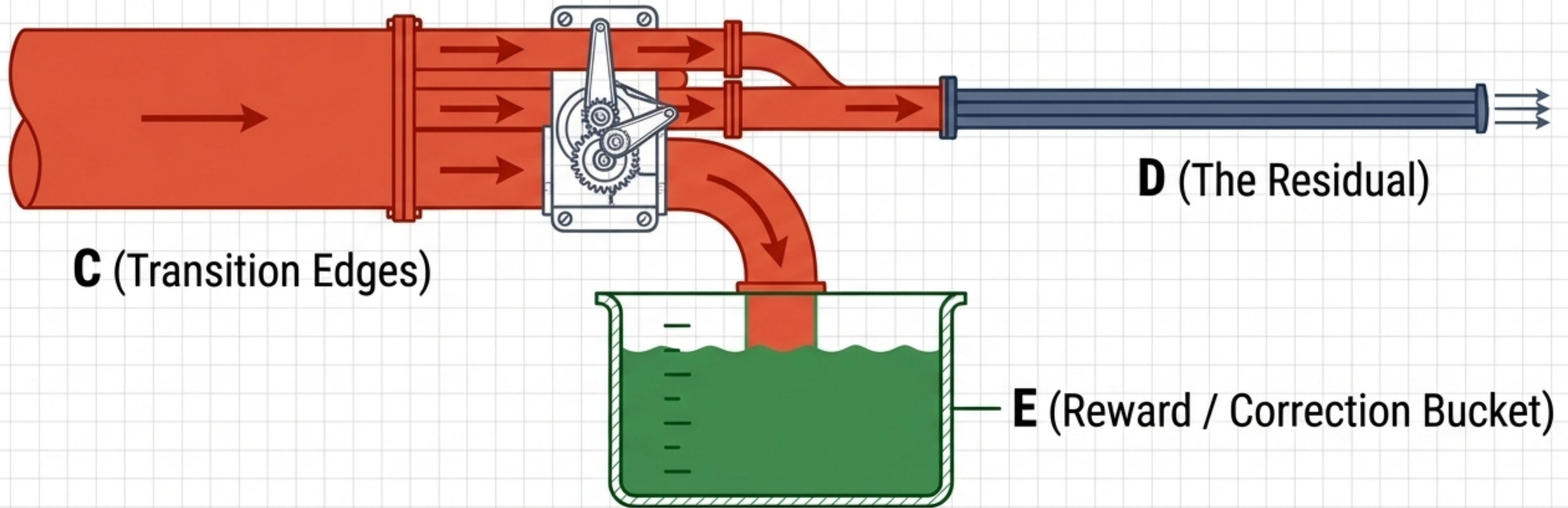
$$p_0 = (I - S)^{-1} \mathbf{e}$$

p_0 = The exact acceptance probability.

$\mathbf{e}$ = Vector indicating accepting final configurations.

Core Insight: The proof doesn't simulate random paths; it directly approximates this inverse matrix in finite, logarithmic space.

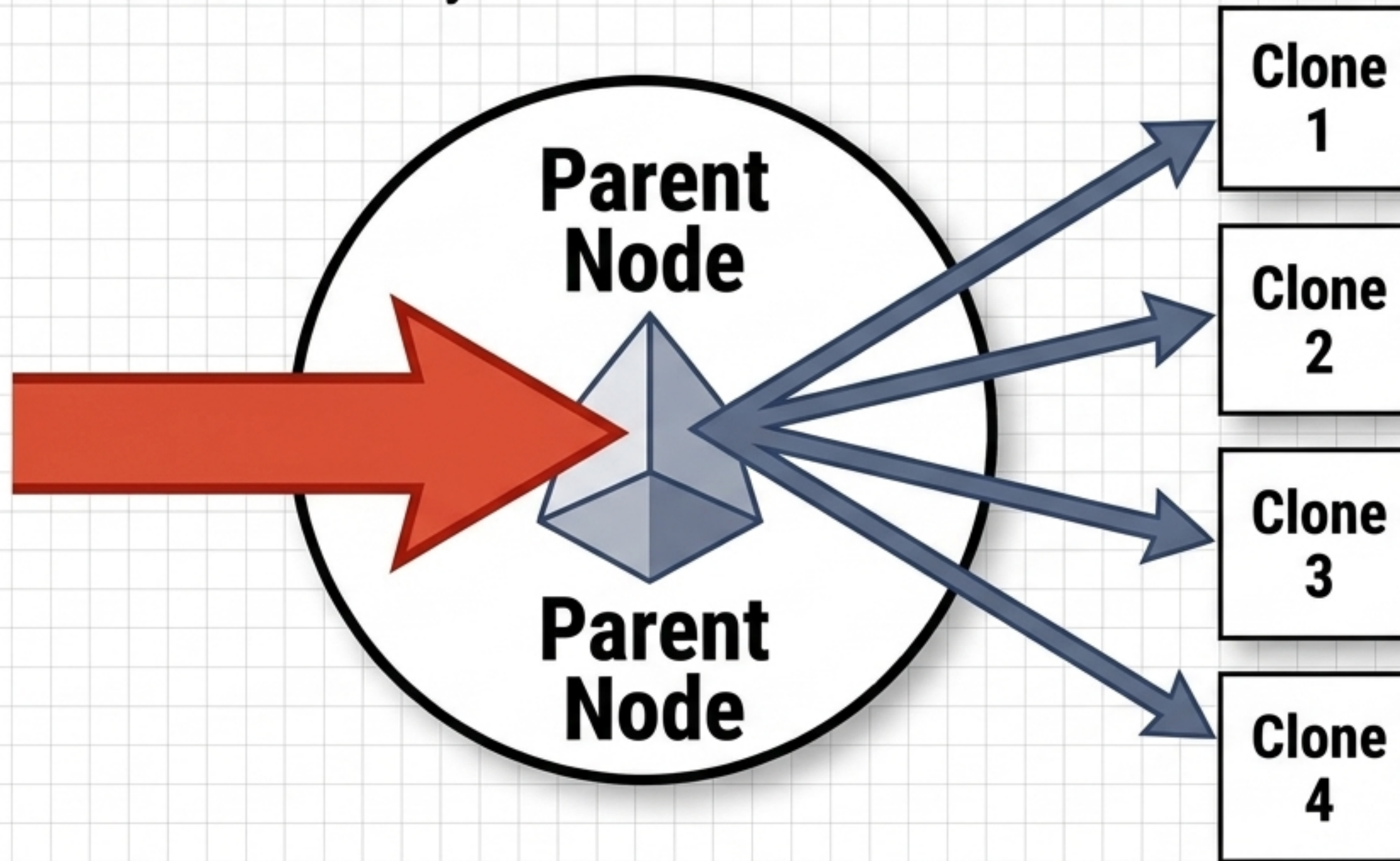
$$\mathbf{I} - \mathbf{D} = (\mathbf{I} + \mathbf{E})(\mathbf{I} - \mathbf{C})$$



- The identity mathematically drains probability mass out of the heavy transition edges (**C**) and deposits it safely into the reward buckets (**E**).
- The leftover residual maze (**D**) becomes exponentially easier to navigate.

The Copy Hierarchy Splitter

Problem: Some probability edges in the maze are too heavy to calculate in small memory.



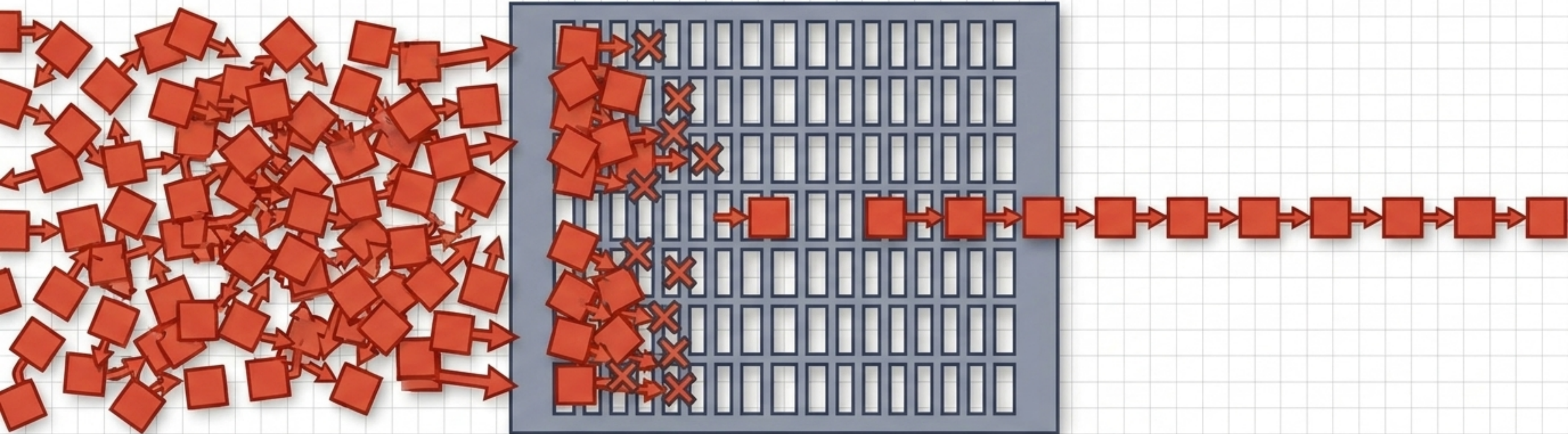
Solution: At each stage, edge weights are divided by 2^H . Splitting large weights across copies adds nodes, but slices the probability into exact, manageable fractions.

Preventing Memory Explosion

Potential Clone Nodes

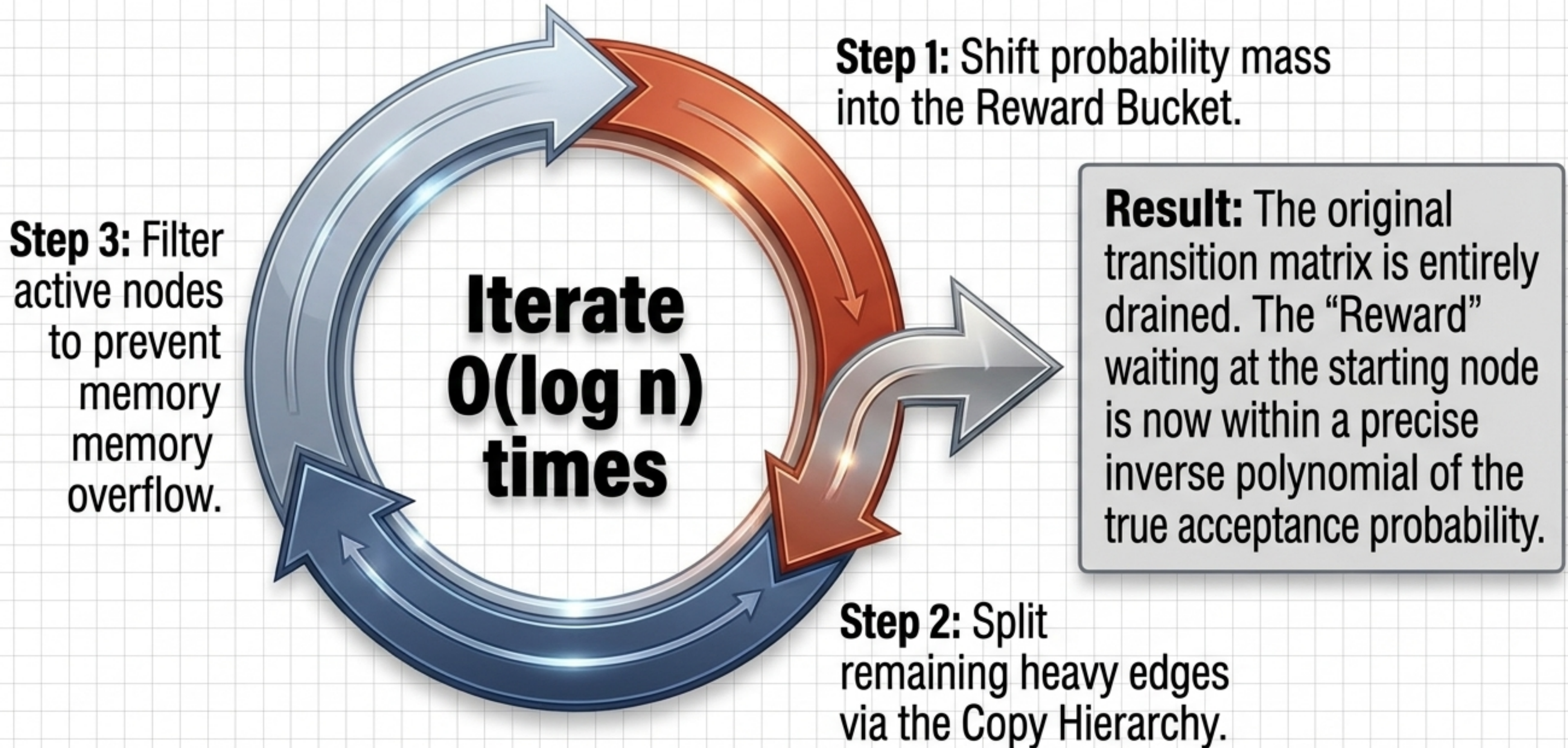
The Counting Filter

Active Nodes



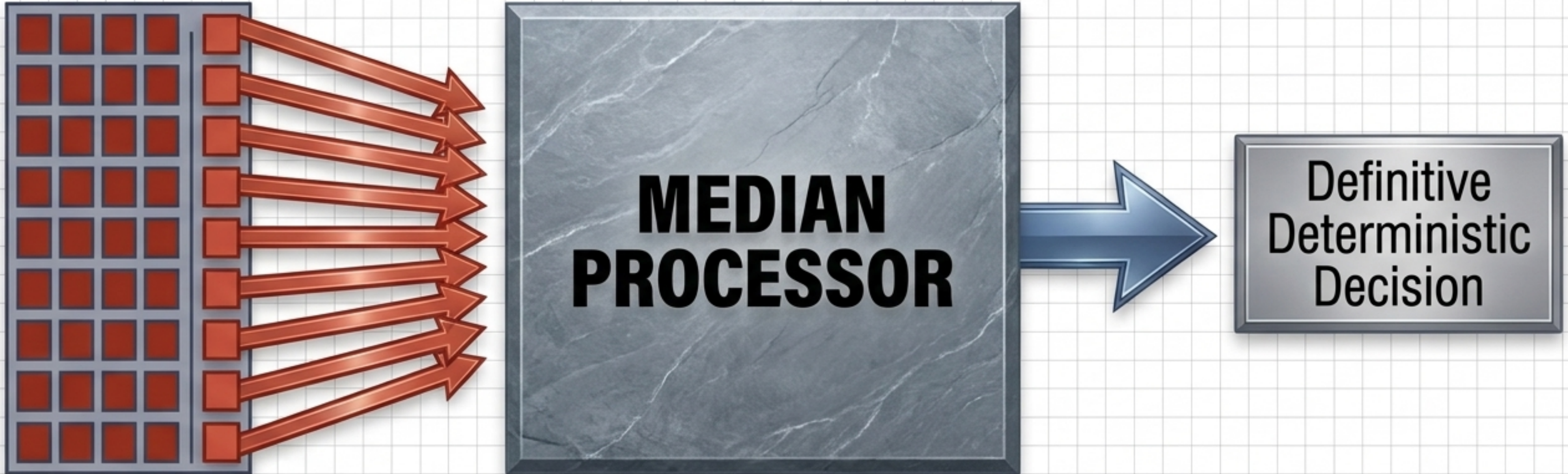
- The proof utilizes a strict ordered-support counting argument.
- Because edge weights are drained into the 'Reward' rapidly, the number of active nodes incident to large weights is strictly controlled.
- Result: Active nodes at most double per stage, completely preventing the exponential explosion of memory.

The Derandomization Engine



The Median Estimator

Polynomially Many Estimates
(Evaluated in $O(\log n)$ memory)



- The hierarchy produces a massive family of estimates. Evaluating them exhaustively takes polynomial time, but only uses $O(\log n)$ memory.
- An estimator using only $O(\log n)$ random bits is mathematically guaranteed to be accurate on more than 75%.
- By taking the median over all choices, the errors are perfectly discarded.

Victory Condition: $L = RL = BPL$

$$L = RL = BPL$$

The deterministic calculation outputs a dyadic fraction:

- If result $\leq 11/24$, it was a NO input.
- If result $\geq 13/24$, it was a YES input.

A simple threshold test at $1/2$ permanently separates the logic logic without ever flipping a real coin. The Bounded-Error guarantees ($\leq 1/3$ and $\geq 2/3$) fit safely inside these margins.

The Boundaries of Computation Have Shifted



Current Status:

- 108-page preprint released September 2026.
- Currently lacks a Lean formalization (machine-verified proof).



The map is still incomplete. Two massive mysteries remain:

- **P vs BPP:** Does randomness help when memory is unlimited?
- **L vs NL:** Can deterministic log space solve directed graph reachability?

The boundaries of computation have shifted, but the labyrinth continues.